

Logging, Debugging and Exceptions

Module 3

1

Logging and Debugging



What problems we're solving

«It crashed in prod, but works locally»

«We saw an error, but don't know the input that caused it»

«The list/map looks wrong, where did it change?»

«Outcome is not what is expected, where it went wrong?»



Logging vs Debugging

Logging	Debugging
Record events over time	Inspect state at specific moment
Works in production	Usually local/dev only
Answers what happened?	Answers why did it happen?

Best practice: Use both together

- Add minimal, high-value logs
- Reproduce the issue locally
- Step through with debugger



```
02-02-2026 17:31:51.260 [INFO ] [] [] [ ] [o.s.d.r.config.RepositoryConfigurationDelegate] Bootstrapping Spring Data
02-02-2026 17:31:59.142 [INFO ] [] [] [ ] [o.s.d.r.config.RepositoryConfigurationDelegate] Finished Spring Data repo
02-02-2026 17:32:02.124 [INFO ] [] [] [ ] [o.s.i.c.DefaultConfiguringBeanFactoryPostProcessor] No bean named 'errorC
02-02-2026 17:32:02.222 [INFO ] [] [] [ ] [o.s.i.c.DefaultConfiguringBeanFactoryPostProcessor] No bean named 'integr
02-02-2026 17:32:05.021 [WARN ] [] [] [ ] [o.s.c.s.PostProcessorRegistrationDelegate$BeanPostProcessorChecker] Bean
02-02-2026 17:32:05.051 [WARN ] [] [] [ ] [o.s.c.s.PostProcessorRegistrationDelegate$BeanPostProcessorChecker] Bean
02-02-2026 17:32:05.066 [WARN ] [] [] [ ] [o.s.c.s.PostProcessorRegistrationDelegate$BeanPostProcessorChecker] Bean
02-02-2026 17:32:06.173 [INFO ] [] [] [ ] [o.s.boot.web.embedded.tomcat.TomcatWebServer] Tomcat initialized with por
02-02-2026 17:32:06.220 [INFO ] [] [] [ ] [org.apache.catalina.core.StandardService] Starting service [Tomcat]
02-02-2026 17:32:06.220 [INFO ] [] [] [ ] [org.apache.catalina.core.StandardEngine] Starting Servlet engine: [Apache
02-02-2026 17:32:06.351 [INFO ] [] [] [ ] [o.a.c.core.ContainerBase.[Tomcat].[localhost].[/]] Initializing Spring en
```



What makes a good log message

A good log answers these questions:

- **What action** (processing order, validating user)
 - **What entity** (userId=123, orderId=456)
 - **What outcome** (success, failed, skipped)
 - **How long** (durationMs=245)
-
- **Good:** "Processing order: orderId=456, items=3, total=99.50, durationMs=120"
 - **Bad:** "Error occurred" or "Processing data" (too vague)
 - **Noisy:** Logging every iteration in a loop at INFO level

Log Levels

Level	When to use	Example
TRACE/DEBUG	Dev-only detail	Variable values in loops
INFO	Business/system milestones	«Order created», «User logged in»
WARN	Unexpected but recoverable	Retry attempt, degraded feature
ERROR	Operation failed; needs attention	Database connection failed

Parametrized Logging

```
//Concatenated
log.info("User " + userId + " created order " + orderId);

//Parametrized
log.info("User {} created order {}", userId, orderId);
```

Benefits:

- Better performance (no string building if log level disabled)
- More readable
- Compatible with structured logging tools



Logging libraries in Java

- **SLF4J** (Simple Logging Facade for Java)
 - API/facade used by most libraries
 - Your code uses this interface
- **Logback**
 - Common backend implementation
 - Preconfigured in Spring Boot
- **Also exists:**
 - `java.util.logging` (built-in, less flexible)
 - **Log4j2** (powerful, but be aware of security/config complexity)

[SLF4J]

[LOGBACK]

Minimal Setup

- **Dependencies** (Maven/Gradle):
 - *slf4j-api* - the interface you code against
 - *logback-classic* - the implementation
- **In frameworks like Spring Boot:**
 - Logging is **preconfigured** out of the box
 - Configuration file: *logback.xml* or *logback-spring.xml*
- **Basic config controls:**
 - Output format (timestamp, level, message)
 - Where logs go (console, file, both)
 - Log level per package



Example

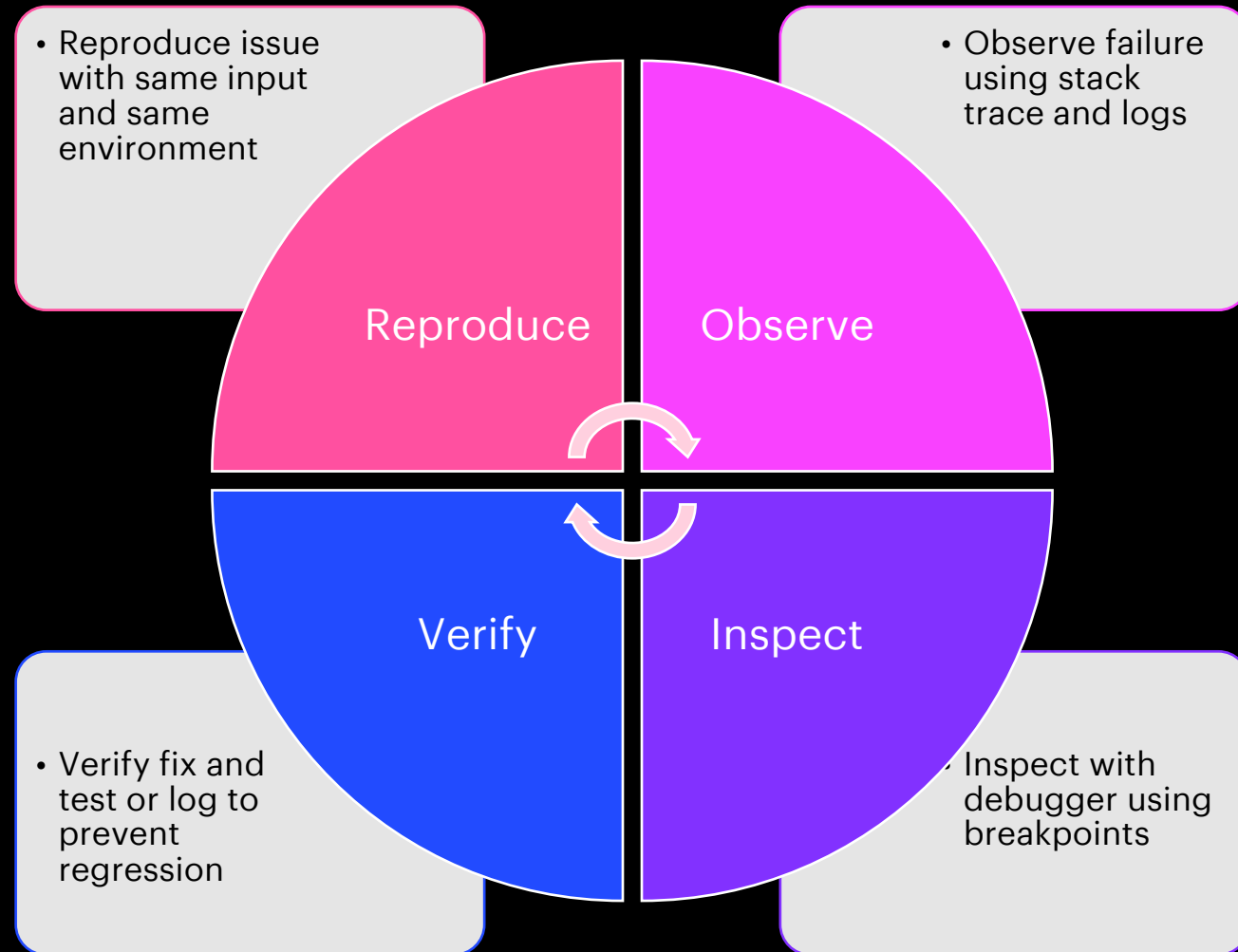
```
private static final Logger log = LoggerFactory.getLogger(OrderService.class); 3 usages
```

 Rename usages

```
public BigDecimal calculateTotalForCustomer(List<Order> orders, String customerId) {  
    log.info("Calculating total: customerId={}, orderCount={}",  
            customerId, orders.size());  
  
    BigDecimal total = orders.stream() Stream<Order>  
        .filter(o -> customerId.equals(o.getCustomerId()))  
        .map(Order::getAmount) Stream<BigDecimal>  
        .reduce(BigDecimal.ZERO, BigDecimal::add);  
  
    log.debug("Calculated total: customerId={}, total={}", customerId, total);  
  
    if (total.compareTo(BigDecimal.ZERO) == 0) {  
        log.warn("Zero total for customer: customerId=", customerId);  
    }  
  
    return total;  
}
```



Debugging



Debugger basics

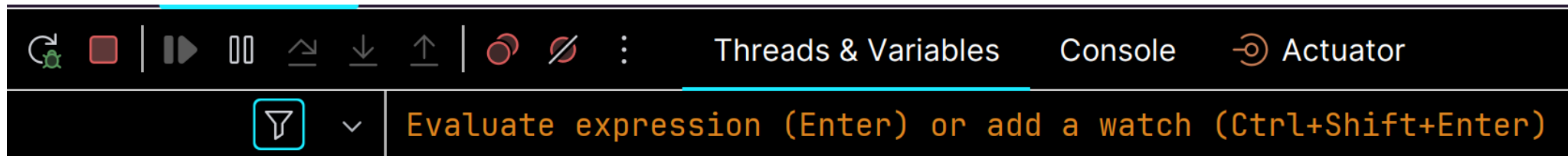
IntelliJ IDEA

- **Key features:**

- **Breakpoints** - pause execution at a line
- **Step Over** (F8) - execute current line, move to next
- **Step Into** (F7) - dive into method call
- **Step Out** (Shift+F8) - finish current method, return to caller

- **Inspection:**

- Variables panel - see all local variables
- Watches - track specific expressions
- Evaluate Expression (Alt+F8) - test code on the fly
- **Pro tip:** Use **conditional breakpoints** for noisy loops
 - Right-click breakpoint →
Condition: `orderId.equals("456")`



<https://www.jetbrains.com/help/idea/debugging-code.html>

Stack traces are clues

When an exception occurs, you get a **stack trace**:

```
java.lang.NullPointerException: Cannot invoke "String.length()"
    at OrderService.processOrder(OrderService.java:45)
    at OrderController.createOrder(OrderController.java:28)
    at java.base/jdk.internal.reflect.NativeMethodAccessorImpl.invoke0(Native Method)
    ...
```

How to read:

1. **Top frames first** (your code) - OrderService.java:45
2. **Exception type + message** - NullPointerException «Cannot invoke "String.length()»»
3. **Connect to last relevant log entry** before crash



Safety in Logging

NEVER log:

- X Passwords, tokens, API keys
- X Credit card numbers (full)
- X Private keys, certificates

Be careful when logging:

- ! Personal data (names, emails, addresses)
- ! Full objects that might contain sensitive fields

Strategies

- ✓ Mask: email=u***@example.com
- ✓ Hash: log hash(userId) for correlation
- ✓ Log IDs only, not full content



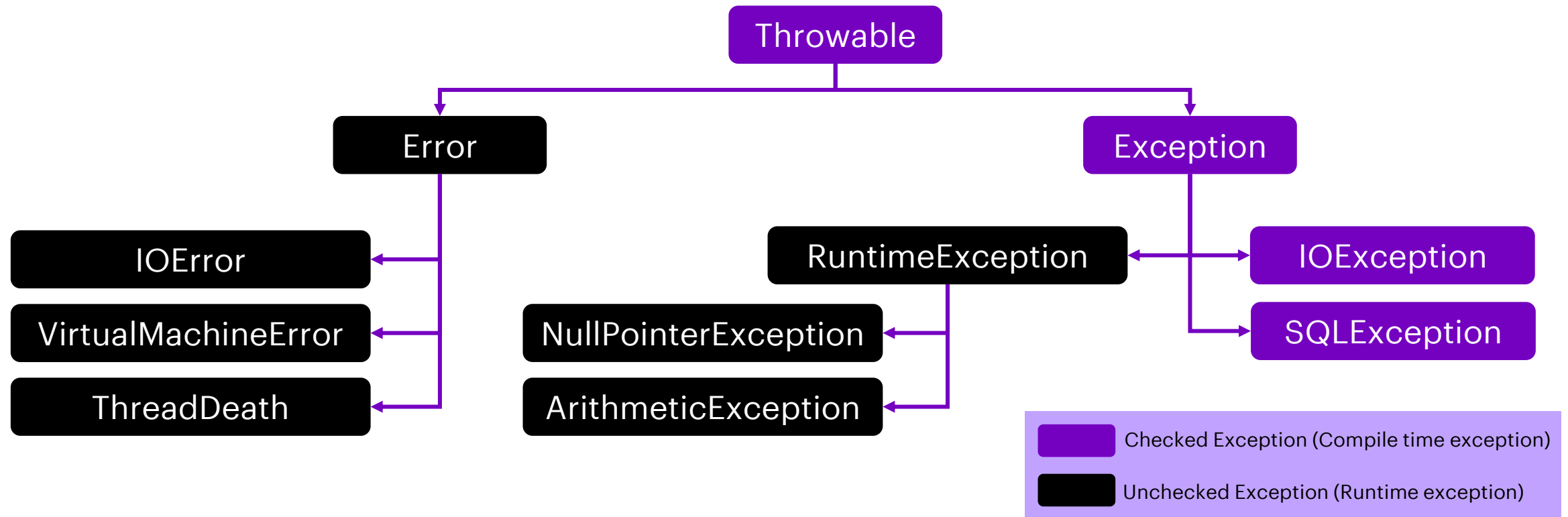
2

Exceptions



Throwable

- The Throwable class is the superclass of all errors and exceptions in the Java language. Only objects that are instances of this class (or one of its subclasses) are thrown by the Java Virtual Machine or can be thrown by the Java **throw** statement. Similarly, only this class or one of its subclasses can be the argument type in a **catch** clause.



Checked vs Unchecked Exceptions

Checked Exceptions

- Are checked by compiler.
- Modern IDEs require that methods, which can throw such exceptions, are either handled with try/catch blocks or throws keyword is added in declaration

```
public void method() throws IOException { no usages
    throw new IOException("This is a sample IOException");
}

public void method() { no usages
    try {
        callSomeMethod(); // Some code that may throw an exception
    } catch (SpecificException e) {
        // Handle specific exception
    } catch (AnotherException e) {
        // Handle another exception
    } finally {
        // Code that will always execute
    }
}
```

Unchecked Exceptions

- Are not checked by compiler, therefore may appear only during runtime.
- Subclasses of `RuntimeException` and `Error` are not checked by compiler

```
// Compiler will not force handling of RuntimeException
public void method() { no usages
    throw new RuntimeException(
        "This is a sample RuntimeException");
}
```



Throwing custom exceptions

- Any amount of custom named exceptions can be created, if they are implementing `Throwable` superclass or extending any of its children.
- Usually, exception handling consists of either simply logging the exception in a structured manner or seeking alternative value to the one that was being retrieved.

```
public class CustomCheckedException extends Exception { 3 usages
    public CustomCheckedException(String message) { 1 usage
        super(message);
    }
}
```

```
// Method that throws the custom checked exception
public void method() throws CustomCheckedException { 1 usage
    throw new CustomCheckedException("This is a custom checked exception");
}

// Method that calls the above method and handles the exception
public void anotherMethod() { no usages
    try {
        method();
    } catch (CustomCheckedException e) {
        log.error("Caught CustomCheckedException: {}", e.getMessage());
    }
}
```

Try-catch

- Any amount of **catch** clauses can be defined after **try** block
- **Catch** clauses are invoked from top to bottom
- Invocation skips following **catch** clauses if a previous **catch** clause was executed

```
public void method() { no usages
    try {
        callSomeMethod(); // Some code that may throw any of the caught exceptions
    } catch (Exception1 e) {
        // Handle Exception1
    } catch (Exception2 | Exception3 e) {
        // Handle Exception2 or Exception3 in the same way
        // Exceptions that are caught together should not have a parent-child relationship
    } finally {
        // Code that will always execute
    }
}
```

Try-with-resources

Support for *try-with-resources* — introduced in Java 7 — allows us to declare resources to be used in a **try** block with the assurance that the resources will be closed after the execution of that block.

The resources declared need to implement the [AutoCloseable](#) interface.

```
Scanner scanner = null;
try {
    scanner = new Scanner(System.in);
    // Use the scanner
} catch (Exception e) {
    System.out.println("Ah well...");
} finally { // Necessary to close the scanner
    if (scanner != null) {
        scanner.close();
    }
}
```

```
try (Scanner scanner = new Scanner(System.in)) {
    // Use the scanner
} catch (Exception e) {
    // Scanner will be closed automatically
    System.out.println("Ah well...");
}
```

Assertions

- `assert` keyword is present since Java 1.4
- Assertion takes a Boolean expression and throws an exception with optional provided text if its false;
- Assertion is disabled by default, for assertions to be executed, java needs to be ran with `-ea` or `-enableassertions` command line argument.

```
assert booleanExpression : "Error message if assertion fails"; // with error message
assert booleanExpression; // without error message
```

```
class MyClass {
    public static void main(String[] args) {
        int x = Integer.parseInt(args[0]); // Convert 1st command-line argument to integer
        assert x >= 0 : "x must be non-negative"; // Assertion to check if x is non-negative
        log.info("Value of x: " + x);
    }
}
```

Input	Result
-2	will throw an AssertionError with message "x must be non-negative"
5	will output "Value of x: 5"



Thank you



Practical task – Smart Home Controller

A Java console app simulating a smart home system with lights, thermostats, and locks.

Your tasks (see README for details):

1. Create a custom exception hierarchy
2. Add validation and exception handling to device methods
3. Set up SLF4J + Logback logging
4. Find and fix 4 planted bugs using the debugger

Full task descriptions and requirements are in the project README.md



Extra material

Logback configuration patterns

Configuration file: src/main/resources/logback.xml

```
<configuration>
  <appender name="CONSOLE" class="ch.qos.logback.core.ConsoleAppender">
    <encoder>
      <pattern>%d{HH:mm:ss.SSS} [%thread] %-5level %logger{36} - %msg%n</pattern>
    </encoder>
  </appender>

  <appender name="FILE" class="ch.qos.logback.core.FileAppender">
    <file>app.log</file>
    <encoder>
      <pattern>%d{yyyy-MM-dd HH:mm:ss} %-5level %logger - %msg%n</pattern>
    </encoder>
  </appender>

  <root level="INFO">
    <appender-ref ref="CONSOLE" />
    <appender-ref ref="FILE" />
  </root>

  <logger name="com.mycompany.myapp" level="DEBUG" />
</configuration>
```

Key elements:

- **Appenders** - where logs go (console, file, database)
- **Pattern** - format (timestamp, level, logger name, message)
- **Root level** - default log level
- **Package-level overrides** - set DEBUG for specific packages

Performance and noise control

Problem: Expensive string building

```
// This builds string even if DEBUG is disabled
logger.debug("Data: " + expensiveToString(data));
```

Solution 1: Parameterized logging

```
logger.debug("Data: {}", data); // toString() only called if needed
```

Solution 2: Guard with level check

```
if (logger.isDebugEnabled()) {
    logger.debug("Data: {}", veryExpensiveComputation());
}
```

Problem: Noise

```
for (Order order : orders) {
    logger.info("Processing order: {}", order); // could be 10k log lines!
}
```



```
logger.info("Processing orders: count={}", orders.size());
// ... process ...
logger.info("Completed orders: success={}, failed={}", successCount, failCount);
```

MDC / Correlation IDs

MDC = Mapped Diagnostic Context

Attach a **correlation ID** (or requestId) to all logs in the current thread:

```
import org.slf4j.MDC;

public void handleRequest(String requestId) {
    MDC.put("requestId", requestId);
    try {
        logger.info("Starting request"); // Will include requestId automatically
        processOrder();
        logger.info("Request completed"); // Will include requestId automatically
    } finally {
        MDC.clear();
    }
}
```

Pattern format:

```
<pattern>%d{HH:mm:ss.SSS} [%X{requestId}] %-5level %logger - %msg%n</pattern>
```

Output:

```
14:23:45.123 [req-456] INFO  OrderService - Starting request
14:23:45.234 [req-456] INFO  OrderService - Request completed
```

Benefits: Trace a single request across multiple classes/methods

Caveat: Be careful with async/multithreading - MDC is thread-local



Common anti-patterns

1. Swallowing exceptions

```
try {
    processOrder(order);
} catch (Exception e) {
    // Silent failure - no one knows this happened!
}
```



```
try {
    processOrder(order);
} catch (Exception e) {
    logger.error("Failed to process order: orderId={}", order.getId(), e);
    // Decide: rethrow, wrap, or handle
}
```

2. Logging and rethrowing repeatedly

```
// Layer 1
catch (Exception e) {
    logger.error("Error in service", e);
    throw e;
}

// Layer 2
catch (Exception e) {
    logger.error("Error in controller", e); // Duplicate stack trace!
    throw e;
}
```

3. Logging huge objects

```
logger.info("Order: {}", entireOrderObjectWithAllFields); // 500-line toString()
```



```
logger.info("Order: orderId={}, total={}", order.getId(), order.getTotal());
```

Sources & useful links

SLF4J

<https://www.slf4j.org/manual.html>

Logback

<https://logback.qos.ch/documentation.html>

Log4j2

<https://logging.apache.org/log4j/2.x/>

IntelliJ IDEA Debugger Basics

<https://www.jetbrains.com/help/idea/debugging-code.html>

Java Logging Best Practices

<https://www.baeldung.com/java-logging-intro>

Structured Logging Guide

<https://reflectoring.io/structured-logging/>

Oracle docs

<https://docs.oracle.com/javase/8/docs/api/java/lang/Throwable.html>

Errors & Exceptions

<https://www.baeldung.com/java-errors-vs-exceptions>

Exception handling

<https://www.baeldung.com/java-exceptions>

Common Exceptions

<https://www.baeldung.com/java-common-exceptions>

Try with Resources

<https://www.baeldung.com/java-try-with-resources>

Java Assertions

<https://www.baeldung.com/java-assert>